

1. Set up

```
In [ ]: # import Python packages
import pandas as pd
import numpy as np
from sklearn import preprocessing
import matplotlib.pyplot as plt

# regression package
import statsmodels.api as sm

# PCA / factor analysis
from sklearn.decomposition import PCA

# clustering packages
from sklearn import cluster
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

In [ ]: # --- factor analysis with scikit-learn -----
# loadings, a varimax rotation, and factor scores.

def varimax(loadings, tol=1e-5, max_iter=500):
    '''Varimax rotation with Kaiser normalization (the standard rule)'''
    L = np.asarray(loadings, dtype=float).copy()
    n_rows, n_cols = L.shape
    if n_cols < 2:
        return L
    h = np.sqrt((L**2).sum(axis=1))          # Kaiser: scale each row to unit
    L = L/h[:,None]
    R, d = np.eye(n_cols), 0.0
    for _ in range(max_iter):
        d_old = d
        basis = L @ R
        diag = np.diag((basis**2).sum(axis=0))
        U, S, Vt = np.linalg.svd(L.T @ (basis**3 - basis @ diag/n_rows))
        R, d = U @ Vt, S.sum()
        if d < d_old*(1+tol):
            break
    return (L @ R)*h[:,None]                # undo the Kaiser scaling

def run_pca(X, n_factors, rotate=False):
    '''Standardize X, run PCA, and return the factor loadings'''
    Z = preprocessing.scale(X)              # PCA on the correlation matrix
    pca = PCA(n_components=n_factors).fit(Z)
    # sklearn reports variances with the (n-1) convention; rescale to get the
    # eigenvalues of the correlation matrix
    eigenvalues = pca.explained_variance_*(len(Z)-1)/len(Z)
    loadings = pca.components_.T*np.sqrt(eigenvalues) # eigenvector x sqrt(e)
    if rotate:
        loadings = varimax(loadings)
    signs = np.sign(loadings.sum(axis=0)); signs[signs==0] = 1
    return loadings*signs                  # orient each factor positively

def factor_scores(X, loadings):
    '''Translate the original data into factor scores (already standardized)'''
    Z = preprocessing.scale(X)
    return Z @ np.linalg.solve(np.corrcoef(Z, rowvar=False), loadings)
```

2. Import Ford Ka Data

```
In [ ]: # read in the data
import os
if not os.path.exists('Ford Ka Data.xlsx'):
    from google.colab import files
    uploaded = files.upload()

FILE_PATH = "Ford Ka Data.xlsx"
SHEET_NAME = "Psychographic Data"

df = pd.read_excel(
    FILE_PATH,
    sheet_name=SHEET_NAME,
    header=6
)

df = df.iloc[:, 1:]

df.head(5)
```

```
Out [ ]:      Q1  Q2  Q3  Q4  Q5  Q6  Q7  Q8  Q9  Q10  ...  Q53  Q54  Q55  Q56  Q57  Q58  Q59
0      6   2   4   3   1   5   5   3   4   4  ...   2   4   5   4   5   3   4
1      7   7   7   5   4   4   5   4   5   5  ...   1   1   1   1   5   4   3
2      5   4   6   5   7   5   3   5   4   5  ...   3   5   6   3   4   4   5
3      4   2   5   4   2   4   5   4   3   4  ...   3   5   4   4   4   2   5
4      5   5   7   6   7   3   4   5   4   2  ...   6   4   5   5   4   5   4
```

5 rows × 62 columns

```
In [ ]: # scale the data

scaler = preprocessing.StandardScaler()
df_sc = scaler.fit_transform(df)

df_sc
```

```
Out [ ]: array([[ 0.58659625, -1.05009144, -0.34514518, ...,  0.16593271,
                  0.24795029, -1.28828394],
               [ 1.23836986,  1.49867419,  1.98691686, ...,  0.89370777,
                  0.24795029,  1.02323031],
               [-0.06517736, -0.03058519,  1.20956285, ..., -0.56184234,
                  0.24795029,  0.25272556],
               ...,
               [ 0.58659625,  0.47916794, -1.1224992 , ..., -2.01739245,
                  -1.30173903, -1.28828394],
               [-1.36872458, -0.54033831,  1.20956285, ...,  0.89370777,
                  1.02279495, -0.51777919],
               [ 1.23836986,  0.98892106, -1.1224992 , ...,  0.89370777,
                  -0.52689437,  1.79373506]])
```

3. PCA Analysis

3.1. PCA - No Rotation

```
In [ ]: df_loadings = run_pca(df_sc,n_factors=62) # run pca with all components

def get_loadings_communalities(loadings,index_names=None,rotated=False,round
```

```

'''Returns a DataFrame containing the loadings and communalities'''
L = np.asarray(loadings)
prefix = 'RC' if rotated else 'PC'          # RC = rotated component, PC = principal component
df = pd.DataFrame(L,
                  index=index_names if index_names is not None else [f'q{i}' for i in range(1,1+L.shape[1])],
                  columns=[f'{prefix}{i}' for i in range(1,1+L.shape[1])])
df['communalities'] = (L**2).sum(axis=1)
return df.round(round_dig)

get_loadings_communalities(df_loadings)

```

```

Out[ ]:

```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	...	PC53	PC54	PC55
q1	-0.545	-0.217	0.569	0.096	-0.005	0.016	0.020	-0.063	0.077	0.029	...	0.08	0.08	0.08
q2	-0.853	0.151	0.288	-0.017	0.023	0.046	0.008	-0.001	0.055	0.013	...	0.02	0.02	0.02
q3	0.175	0.660	-0.174	0.011	0.086	-0.004	0.068	0.039	-0.034	-0.131	...	-0.02	-0.02	-0.02
q4	-0.061	0.617	-0.569	-0.088	-0.039	0.009	0.104	0.018	0.027	-0.024	...	0.04	0.04	0.04
q5	-0.174	0.799	0.261	0.033	-0.017	0.035	0.042	-0.008	0.004	-0.029	...	0.04	0.04	0.04
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
q58	0.234	-0.101	0.627	-0.038	-0.007	-0.085	0.094	-0.020	-0.087	0.027	...	0.00	0.00	0.00
q59	0.208	-0.210	0.609	-0.001	-0.123	0.090	0.065	0.202	0.112	0.034	...	-0.02	-0.02	-0.02
q60	-0.252	0.037	-0.648	0.079	-0.017	0.081	-0.071	0.087	-0.062	-0.004	...	-0.02	-0.02	-0.02
q61	-0.401	0.130	-0.600	0.007	0.090	-0.026	0.110	-0.137	0.119	0.070	...	0.02	0.02	0.02
q62	-0.280	0.143	-0.545	-0.096	0.059	0.039	-0.005	0.041	0.067	-0.105	...	0.00	0.00	0.00

62 rows x 63 columns

```

In [ ]:
def get_summary(loadings,rotated=False,round_dig=2):
    ''' Print a summary of the PCA fit '''
    L = np.asarray(loadings)
    prefix = 'RC' if rotated else 'PC'
    ss = (L**2).sum(axis=0)
    n_vars = L.shape[0]
    return pd.DataFrame([ss,ss/n_vars,np.cumsum(ss)/n_vars],
                        columns=[f'{prefix}{i}' for i in range(1,1+L.shape[1])],
                        index=['Sum of Squared Loadings','Proportion of Variance Explained',
                              'Cumulative Proportion']).round(round_dig)

get_summary(df_loadings)

```

```

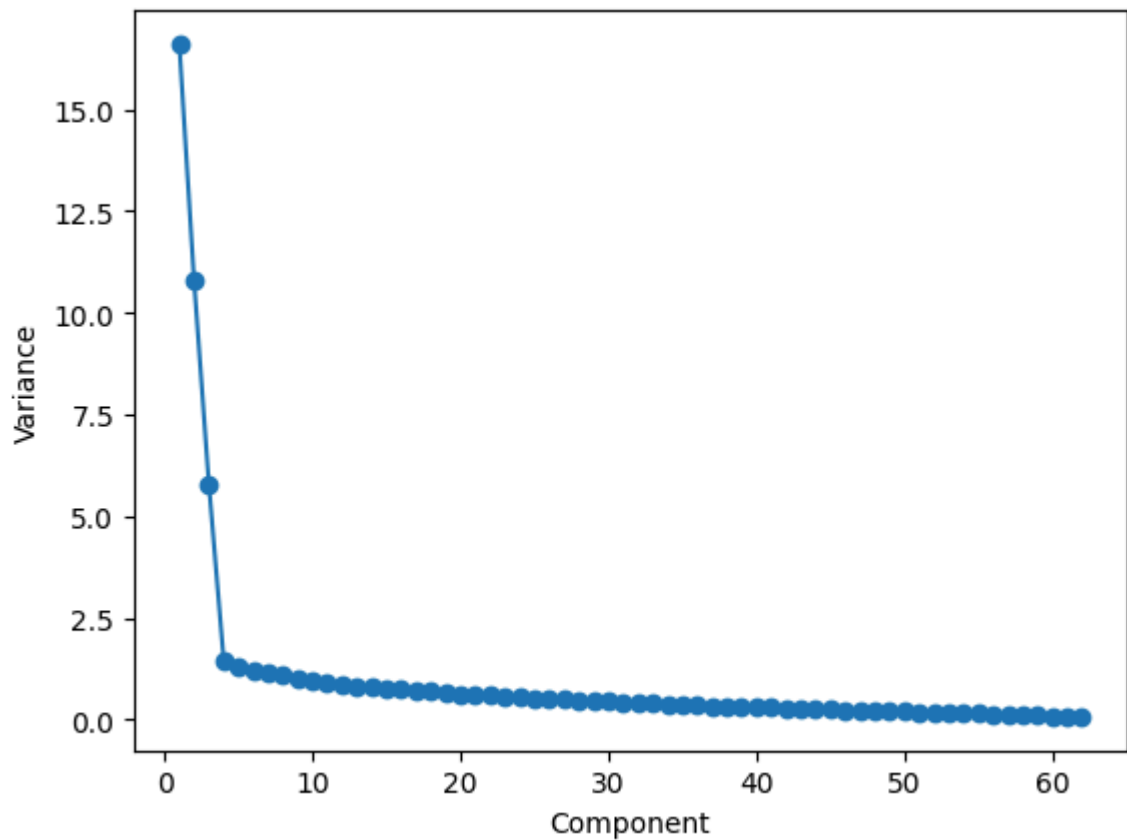
Out[ ]:

```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9	PC10	...	PC53	PC54	PC55
Sum of Squared Loadings	16.60	10.78	5.79	1.47	1.28	1.21	1.17	1.12	1.01	0.98	...	0.16	0.15	0.15
Proportion of Variance Explained	0.27	0.17	0.09	0.02	0.02	0.02	0.02	0.02	0.02	0.02	...	0.00	0.00	0.00
Cumulative Proportion	0.27	0.44	0.54	0.56	0.58	0.60	0.62	0.64	0.65	0.67	...	0.98	0.99	0.99

3 rows x 62 columns

```
In [ ]: # scree plot - shows successive proportion of additional variance that each
eigenvalues = (df_loadings**2).sum(axis=0)
plt.plot(1+np.arange(len(eigenvalues)),eigenvalues,'o-')
plt.xlabel('Component')
plt.ylabel('Variance')
plt.show()
```



3.2. PCA-Varimax Rotation

```
In [ ]: df_loadings_rotated = run_pca(df_sc, n_factors= 3, rotate=True)
get_loadings_communalities(df_loadings_rotated, rotated=True)
```

```
Out[ ]:
```

	RC1	RC2	RC3	communalities
q1	0.630	-0.322	0.409	0.668
q2	0.910	0.023	0.075	0.834
q3	-0.119	0.687	-0.101	0.497
q4	0.008	0.643	-0.544	0.708
q5	0.340	0.751	0.240	0.737
...	...	...	...	...
q58	-0.089	-0.115	0.661	0.458
q59	-0.084	-0.224	0.633	0.458
q60	0.093	0.050	-0.688	0.484
q61	0.260	0.120	-0.675	0.537
q62	0.159	0.144	-0.592	0.396

62 rows x 4 columns

```
In [ ]: df_scores = factor_scores(df_sc, df_loadings_rotated)
df_scores = pd.DataFrame(df_scores, columns=['RC1', 'RC2', 'RC3'])
df_scores.head(5)
```

```
Out[ ]:
```

	RC1	RC2	RC3
0	-1.002237	-1.042724	-0.706512
1	1.562683	-0.003871	-0.528081
2	-0.264483	1.857548	0.225793
3	-1.093728	-0.691215	-1.278540
4	-0.526620	1.718462	-0.033414

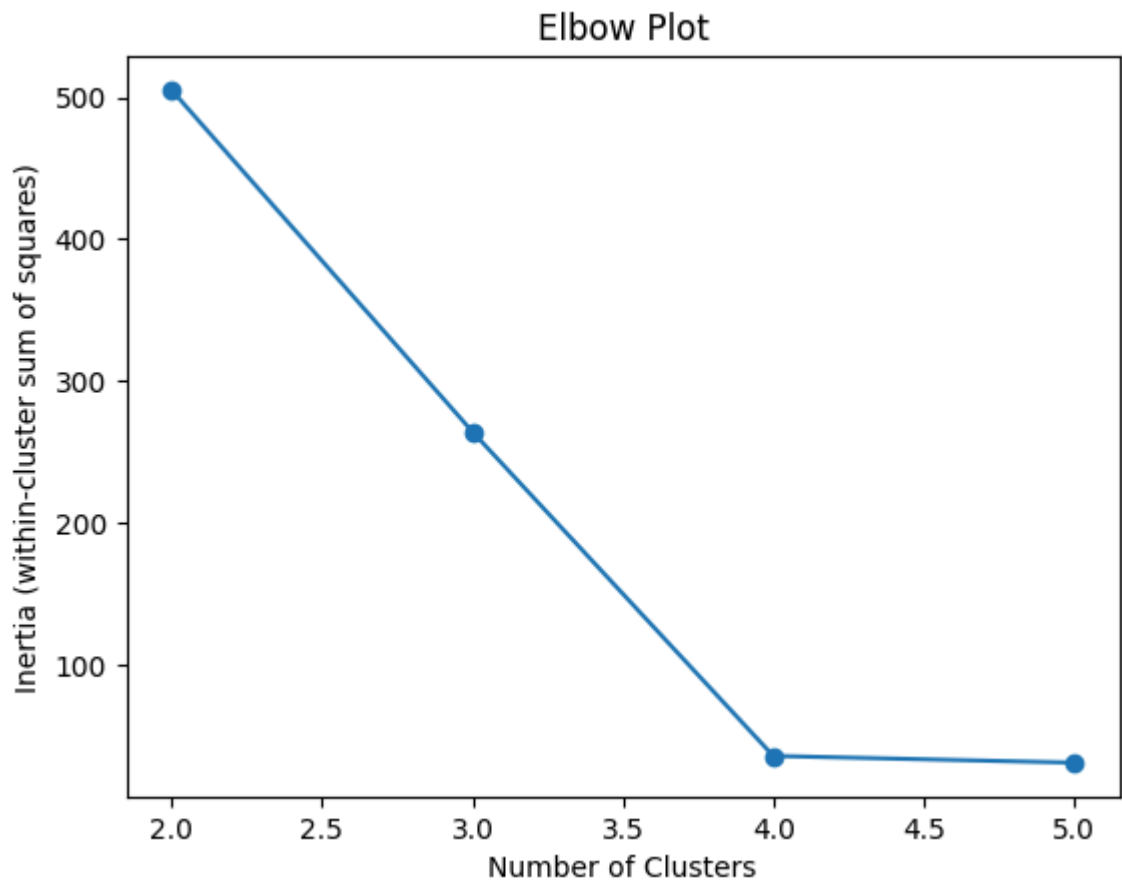
4. Cluster Analysis - K-Means

```
In [ ]: def check_clusters(data, labels):
    print(pd.Series(labels).value_counts().sort_index().to_dict())

    return pd.pivot_table(data, index=labels)
```

```
In [ ]: # Elbow plot: inertia (within-cluster sum of squares) against the number of
# We compute it directly rather than using an external package.
# 2 to 5 segments
ks = range(2,6)
inertias = [cluster.k_means(df_scores, n_clusters=k, n_init=50, random_state=10)

plt.plot(list(ks), inertias, marker="o")
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia (within-cluster sum of squares)')
plt.title('Elbow Plot')
plt.show()
```



```
In [ ]: centroids_km, labels_km, inertia_km = cluster.k_means(df_scores, n_clusters=4,
order          = np.argsort(centroids_km[:,2]))
labels_km      = np.argsort(order)[labels_km]
centroids_km   = centroids_km[order]

check_clusters(df_scores, labels_km)
```

```
{0: 75, 1: 78, 2: 65, 3: 32}
```

```
Out[ ]:
```

	RC1	RC2	RC3
0	-1.066228	-0.748296	-0.722523
1	1.383266	-0.430916	-0.245159
2	-0.200742	1.643329	-0.056442
3	-0.464984	-0.533837	2.405634

```
In [ ]: # choose K = 4 clusters based on elbow plot
kmeans = KMeans(
    n_clusters=4,
    random_state=1990,
    n_init=50
)
```

```
labels = kmeans.fit_predict(df_scores)
check_clusters(df_scores, labels)
```

```
{0: 32, 1: 65, 2: 78, 3: 75}
```

```
Out[ ]:
```

	RC1	RC2	RC3
0	-0.464984	-0.533837	2.405634
1	-0.200742	1.643329	-0.056442
2	1.383266	-0.430916	-0.245159
3	-1.066228	-0.748296	-0.722523

Chi-Square Test

```
In [ ]: psy_centroids, psy_labels, psy_inertia = cluster.k_means(df_scores, n_clusters=4,
df_psy_cluster = pd.DataFrame({
    'Preference Group': selftest['Preference Group'].values,
    'Cluster': psy_labels
}))

# Keep only Preference Group 1 and 2
df_psy_cluster = df_psy_cluster[
    df_psy_cluster['Preference Group'].isin([1, 2])
].copy()

def cluster_cross_tab(df_psy_cluster):
    print('\n=====')
    print('\n===== Cluster vs Preference Group =====')
    # Count
    count_table = pd.crosstab(
        df_psy_cluster['Cluster'],
        df_psy_cluster['Preference Group']
    )
    # Percentage within each Cluster
    percent_table = pd.crosstab(
        df_psy_cluster['Cluster'],
        df_psy_cluster['Preference Group'],
        normalize='index'
    ) * 100
    combined_table = pd.DataFrame({
        '1 Count': count_table[1],
        '2 Count': count_table[2],
        '1 %': percent_table[1],
        '2 %': percent_table[2]
    })
    print(combined_table.round(1))
    print('\n=====')
    return

def cluster_chi2_test(df_psy_cluster):
    table = pd.crosstab(
        df_psy_cluster['Cluster'],
        df_psy_cluster['Preference Group']
    )
    chi2, p, dof, expected = chi2_contingency(table)
    print('\n=====')
    print('\n===== Cluster vs Preference Group =====')
    print(f'Chi2 = {chi2:.3f}')
    print(f'p = {p:.4f}')
    print(f'dof = {dof}')
    print('\n=====')
    return
```

```
cluster_cross_tab(df_psy_cluster)
cluster_chi2_test(df_psy_cluster)
```

```
=====
```

```
===== Cluster vs Preference Group =====
```

	1 Count	2 Count	1 %	2 %
Cluster				
0	18	4	81.8	18.2
1	29	32	47.5	52.5
2	34	13	72.3	27.7
3	35	23	60.3	39.7

```
=====
```

```
=====
```

```
===== Cluster vs Preference Group =====
```

```
Chi2 = 11.240
```

```
p = 0.0105
```

```
dof = 3
```

```
=====
```